

SQL

CLÁUSULAS AVANZADAS

JOINS

UNIR COLUMNAS DE VARIAS TABLAS

JOINS

Si observamos la tabla `matriculas`.

MATRICULAS				
matricula_id	fecha	precio_pagado	alumno_id	curso_id
1	44951	45.00	1	1
2	44969	50.00	2	1
3	44990	195.00	1	3

Vemos que tenemos alguna información pero no toda, porque tenemos muchos `id`.

Podemos hacer peticiones juntando varias tablas usando la cláusula

JOIN

JOINS

INNER JOINS

JOINS

INNER JOINS

Los **INNER JOINS** son las uniones POR DEFECTO.

```
-- Seleccionar datos de la tabla matriculas y la tabla cursos.
```

```
SELECT *  
FROM matriculas  
INNER JOIN alumnos  
    ON matriculas.alumno_id = alumnos.alumno_id;
```

```
/*
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+  
| matricula_id | fecha       | precio_pagado | alumno_id | curso_id | alumno_id | nombre | p  
+-----+-----+-----+-----+-----+-----+-----+-----+  
|           1 | 2023-01-25 |         45.00 |          1 |          1 |          1 | Jorge | I  
|           2 | 2023-02-12 |         50.00 |          2 |          1 |          2 | Javier | M  
|           3 | 2023-03-05 |        195.00 |          1 |          3 |          1 | Jorge | I  
+-----+-----+-----+-----+-----+-----+-----+-----+
```

JOINS

INNER JOINS

Si tenemos alguna columna repetida en las dos tablas **TENDREMOS QUE DECIRLE DE QUÉ TABLA LO COGEMOS.**

```
-- Seleccionar columnas que se repiten en la tabla matriculas y la tabla cursos.

SELECT matricula_id, alumnos.alumno_id ,precio_pagado, nombre, primer_apellido
FROM matriculas
INNER JOIN alumnos
    ON matriculas.alumno_id = alumnos.alumno_id;

/*
+-----+-----+-----+-----+-----+
| matricula_id | alumno_id | precio_pagado | nombre | primer_apellido |
+-----+-----+-----+-----+-----+
|          1 |          1 |          45.00 | Jorge  | López           |
|          2 |          2 |          50.00 | Javier | Martínez        |
|          3 |          1 |         195.00 | Jorge  | López           |
+-----+-----+-----+-----+-----+*/
```

JOINS

INNER JOINS

Nos permite ponerle alias a los nombres de las tablas para que no sea tan largo. LO SOLEMOS ABREVIAR A UNA SOLA LETRA

ALIAS DE TABLA

```
-- Abreviar los nombres de las tablas.

SELECT matricula_id, a.alumno_id ,precio_pagado, nombre, primer_apellido
FROM matriculas m
INNER JOIN alumnos a
    ON m.alumno_id = a.alumno_id;

/*
+-----+-----+-----+-----+-----+
| matricula_id | alumno_id | precio_pagado | nombre | primer_apellido |
+-----+-----+-----+-----+-----+
|          1 |          1 |          45.00 | Jorge  | López           |
|          2 |          2 |          50.00 | Javier | Martínez        |
|          3 |          1 |         195.00 | Jorge  | López           |
+-----+-----+-----+-----+-----+*/
```

JOINS

INNER JOINS

EJERCICIO 01

- Coge la tabla de `items_pedido` de la BBDD `tienda`:
 - Y la juntas con la tabla `productos`
 - Y muestra solo las columnas `pedido_id`, `producto_id`, `nombre` y `cantidad_stock`.

JOINS

INNER JOINS

JUNTAR MÁS DE DOS TABLAS

```
-- Juntar las tablas `matriculas`, `alumnos` y `cursos`.

SELECT matricula_id, precio_pagado, a.nombre AS nombre_alumno, c.nombre AS curso
FROM matriculas m
INNER JOIN alumnos a
    ON a.alumno_id = m.alumno_id
INNER JOIN cursos c
    ON c.curso_id = m.curso_id;

/*
+-----+-----+-----+-----+
| matricula_id | precio_pagado | nombre_alumno | curso |
+-----+-----+-----+-----+
|          1 |         45.00 | Jorge         | HTML  |
|          2 |         50.00 | Javier        | HTML  |
|          3 |        195.00 | Jorge         | JS    |
+-----+-----+-----+-----+*/
```

Fíjate en que he puesto **ALIAS** a las columnas porque si no nos liamos mucho.

JOINS

INNER JOINS

EJERCICIO 02

- Coge la tabla de `pagos` de la BBDD `facturacion`:
 - Y la juntas con la tabla `metodos_pago` y la de `clientes`
 - Y muestra solo las columnas `fecha`, `factura_id`, `cantidad`, `nombre` y `metodo_pago`.

JOINS

OUTER JOINS

JOINS

OUTER JOINS

Cuando hacemos un **JOIN** normal, nunca se van a mostrar los resultados de aquellos registros que no cumplan una condición.

```
-- Juntar las tablas `matriculas`, `alumnos`.
```

```
SELECT a.alumno_id, nombre, precio_pagado
FROM alumnos a
JOIN matriculas m
      ON m.alumno_id = a.alumno_id;
```

```
/*
+-----+-----+-----+
| alumno_id | nombre | precio_pagado |
+-----+-----+-----+
|          1 | Jorge  |          45.00 |
|          2 | Javier |          50.00 |
|          1 | Jorge  |         195.00 |
+-----+-----+-----+*/
```

Yo sé que hay un alumno más que no aparece porque no cumple la condición de que `ON m.alumno_id = a.alumno_id`.

JOINS

OUTER JOINS

LEFT JOIN

```
-- Juntar las tablas `matriculas`, `alumnos` mostrando  
-- todos los datos de la tabla de la "izquierda" (la primera tabla).
```

```
SELECT a.alumno_id, nombre, precio_pagado  
FROM alumnos a  
LEFT JOIN matriculas m  
      ON m.alumno_id = a.alumno_id;
```

```
/*  
+-----+-----+-----+  
| alumno_id | nombre | precio_pagado |  
+-----+-----+-----+  
|          1 | Jorge  |          195.00 |  
|          1 | Jorge  |           45.00 |  
|          2 | Javier |           50.00 |  
|          3 | Sarah  |           NULL  |  
+-----+-----+-----+*/
```

La versión **RIGHT JOIN** hace exactamente lo mismo, pero cogiendo los datos de la “derecha” (la segunda tabla).

JOINS

CLAUSULA USING

Si coincide que hacemos un **JOIN** y que las dos columnas por las que queremos juntar las tablas se llaman igual, podemos usar la cláusula **USING** en su lugar.

```
-- Juntar las tablas `matriculas` y `alumnos`
```

```
SELECT a.alumno_id, nombre, precio_pagado
FROM alumnos a
LEFT JOIN matriculas m
      USING (alumno_id);
```

```
/*
+-----+-----+-----+
| alumno_id | nombre | precio_pagado |
+-----+-----+-----+
|          1 | Jorge  |          195.00 |
|          1 | Jorge  |           45.00 |
|          2 | Javier |           50.00 |
|          3 | Sarah  |           NULL  |
+-----+-----+-----+*/
```

UNIONS

UNIR FILAS DE VARIAS TABLAS

UNIONS

Podemos unir más filas de otras tablas si combinamos una **QUERY** con otra usando la cláusula **UNION**

```
-- Añadir las filas de la tabla `alumnos` a las de la tabla `profesores`

SELECT nombre, primer_apellido
FROM profesores
UNION
SELECT nombre, primer_apellido
FROM alumnos;

/*
+-----+-----+
| nombre | primer_apellido |
+-----+-----+
| Ivan   | Luengo          |
| Samuel | Rodríguez       |
| Berto  | Yanez           |
| Juan   | Perez           |
| Pedro  | Garcia          |
| Maria  | López           |
+-----+-----+*/
```

UNIONS

Por ejemplo, podemos unir la misma tabla varias veces añadiendo una columna extra que defina un estado distinto para cada unión.

Ej: Todos los pedidos que sean de este año que tengan un estado **ACTIVO** y todos los pedidos anteriores que tengan un estado **ANTIGUO**

```
-- Duplicar las filas de la tabla `productos`.

SELECT pedido_id, fecha_pedido, 'ACTIVO' AS estado
FROM pedidos
WHERE fecha_pedido >= '2023-01-01'
UNION
SELECT pedido_id, fecha_pedido, 'ANTIGUO' AS estado
FROM pedidos
WHERE fecha_pedido < '2023-01-01'
LIMIT 3;
```

```
/*
+-----+-----+-----+
| pedido_id | fecha_pedido | estado |
+-----+-----+-----+
|          1 | 2023-01-30   | ACTIVO |
|          2 | 2022-08-02   | ANTIGUO |
|          3 | 2021-12-01   | ANTIGUO |
+-----+-----+-----+*/
```

UNIONS

EJERCICIO 01

- Coge la tabla de `consumidores` de la BBDD `tienda`:
 - Y usando **UNIONS** muestra las columnas `nombre`, `apellidos`, `ciudad`, `puntos` y una columna extra que defina si el consumidor es de tipo **ORO**, **PLATA**, **BRONCE** o **NORMAL**.
 - Si tiene más de 3000 puntos será **ORO**
 - Si tiene más de 2000 puntos será **PLATA**
 - Si tiene más de 1000 puntos será **BRONCE**
 - Si tiene menos de 1000 puntos será **NORMAL**

FUNCIONES DE AGREGACIÓN

MAX() MIN()

SUM() AVG()

COUNT ()

FUNCIONES DE AGREGACIÓN

FUNCIÓN MAX()

De todas los registros de una tabla, te devuelve el más **GRANDE**.

```
-- Valor más alto de la tabla de `facturas`.

SELECT MAX(factura_total) AS factura_mas_alta
FROM facturas;

/*
+-----+
| factura_mas_alta |
+-----+
|          189.12 |
+-----+*/
```

FUNCIONES DE AGREGACIÓN

FUNCIÓN MIN()

De todas los registros de una tabla, te devuelve el más **PEQUEÑO**.

```
-- Valor más bajo de la tabla de `facturas`.
```

```
SELECT MIN(factura_total) AS factura_mas_baja  
FROM facturas;
```

```
/*  
+-----+  
| factura_mas_baja |  
+-----+  
|          101.79 |  
+-----+*/
```

FUNCIONES DE AGREGACIÓN

FUNCIÓN SUM()

Suma todos los registros de esa columna.

```
-- Valor total de todas las `facturas`.

SELECT SUM(factura_total) AS facturacion_total
FROM facturas;
```

```
/*
+-----+
| facturacion_total |
+-----+
|          2590.60 |
+-----+*/
```

FUNCIONES DE AGREGACIÓN

FUNCIÓN 🙌 AVG()

Calcula la media de todos los valores de la columna.

```
-- Valor medio de todas las `facturas`.

SELECT AVG(factura_total) AS facturacion_media
FROM facturas;
```

```
/*
+-----+
| facturacion_media |
+-----+
|      152.388235   |
+-----+*/
```

FUNCIONES DE AGREGACIÓN

FUNCIÓN COUNT()

Cuenta el número de registros de una columna.

```
-- Cuantas facturas hay..  
  
SELECT count(factura_total) AS numero_de_facturas  
FROM facturas;
```

```
/*  
+-----+  
| numero_de_facturas |  
+-----+  
|                  17 |  
+-----+*/
```

¡Ojo! porque solo contará los registros que tengan valor, no los que tengan **null**. Así que para contar todos los registros habrá que usar **COUNT(*)**

FUNCIONES DE AGREGACIÓN

RESUMEN

```
SELECT
    MAX(factura_total) AS mas_alta,
    MIN(factura_total) AS mas_baja,
    SUM(factura_total) AS facturacion_total,
    AVG(factura_total) AS factura_media,
    count(factura_total) AS numero_de_facturas
FROM facturas;
```

```
/*
+-----+-----+-----+-----+-----+
| mas_alta | mas_baja | facturacion_total | factura_media | numero_de_facturas |
+-----+-----+-----+-----+-----+
| 189.12 | 101.79 | 2590.60 | 152.388235 | 17 |
+-----+-----+-----+-----+-----+*/
```

AGRUPAR AGREGACIONES

GROUP BY

AGRUPAR AGREGACIONES

CLÁUSULA GROUP BY

Cuando usamos funciones de agregación podemos decirle a SQL que esa agregación la haga por cada uno de los valores de una columna. Por ejemplo: Hazme la suma total de las facturas DE CADA USUARIO.

```
-- Cuanto ha gastado cada cliente.  
  
SELECT cliente_id, SUM(factura_total) AS total_cliente  
FROM facturas  
GROUP BY cliente_id;
```

```
/*  
+-----+-----+  
| cliente_id | total_cliente |  
+-----+-----+  
|          1 |         802.89 |  
|          2 |         101.79 |  
|          3 |         705.90 |  
|          5 |         980.02 |  
+-----+-----+*/
```

AGRUPAR AGREGACIONES

**GROUP BY
HAVING**

AGRUPAR AGREGACIONES

CLÁUSULA HAVING

Una vez hemos agrupado algo usando **GROUP BY**, podemos filtrar los resultados que da esa agrupación, **pero no podemos usar el WHERE**, necesitamos usar el **HAVING**

```
-- Cuanto ha gastado cada cliente, y solo muestra los que han gastado más de 300.
```

```
SELECT cliente_id, SUM(factura_total) AS total_cliente
FROM facturas
GROUP BY cliente_id
HAVING total_cliente > 300;
```

```
/*
+-----+-----+
| cliente_id | total_cliente |
+-----+-----+
|          1 |         802.89 |
|          3 |         705.90 |
|          5 |         980.02 |
+-----+-----+*/
```

OJO CON EL ORDEN OTRA VEZ

A riesgo de parecer cansino, me gustaría repetir que mucho cuidado con el orden de las cláusulas.

```
SELECT
    factura_id,
    factura_total,
    pago_total,
    factura_fecha,
    nombre,
    ciudad,
    SUM(factura_total) AS total_por_cliente
FROM facturas
JOIN clientes USING (cliente_id)
WHERE factura_fecha BETWEEN '2019-01-01' AND '2019-05-31'
GROUP BY nombre
HAVING pago_total > 0
ORDER BY pago_total
LIMIT 1;
```

/*

factura_id	factura_total	pago_total	factura_fecha	nombre	ciudad	total_por_cliente
18	180.17	42.77	2019-05-23	Susana	Llança	1

AGRUPACIONES

EJERCICIO 01

- Coge la tabla de `consumidores` de la BBDD `tienda` :
 - Genera un listado de las puntuaciones más altas **POR COMUNIDAD AUTÓNOMA**.
 - Ordena el resultado por puntos de mayor a menor.
 - Muestra solo los resultados que tengan más de 1000 puntos.

SUBCONSULTAS

BUSCAR ALGO UTILIZANDO MÁS DE UN SELECT

SUBCONSULTAS

A veces tenemos que hacer consultas que son más complejas y necesitamos utilizar la cláusula **SELECT** más veces. Veamos un ejemplo.

Ej: Busca todos los cursos que sean más caros **que el curso de CSS** {font-sm} DIVIDE EL PROBLEMA EN VARIOS PROBLEMILLAS

1. ¿Cuánto vale el curso de CSS?

```
-- Cuanto vale el curso de CSS.
```

```
SELECT precio
FROM cursos
WHERE curso_id = 2;
```

```
/*
+-----+
| precio |
+-----+
|  50.00 |
+-----+*/
```

SUBCONSULTAS

Ej: Busca todos los cursos que sean más caros **que el curso de CSS** {font-sm} DIVIDE EL PROBLEMA EN VARIOS PROBLEMILLAS

2. ¿Cómo obtengo todos los cursos más caros de 50?

```
-- Cursos más caros de 50.
```

```
SELECT curso_id, nombre, precio
FROM cursos
WHERE precio > 50;
```

```
/*
+-----+-----+-----+
| curso_id | nombre | precio |
+-----+-----+-----+
|         4 | JS     | 200.00 |
|         5 | Node.js | 250.00 |
+-----+-----+-----+*/
```

SUBCONSULTAS

Ej: Busca todos los cursos que sean más caros **que el curso de CSS** {font-sm} DIVIDE EL PROBLEMA EN VARIOS PROBLEMILLAS

3. Mezcla los dos resultados poniendo todo el **SELECT** en el lugar del 50

```
-- Cursos más caros de 50.

SELECT curso_id, nombre, precio
FROM cursos
WHERE precio > (
    SELECT precio
    FROM cursos
    WHERE curso_id = 2
);

/*
+-----+-----+-----+
| curso_id | nombre | precio |
+-----+-----+-----+
|         4 | JS     | 200.00 |
|         5 | Node.js | 250.00 |
+-----+-----+-----+*/
```



SUBCONSULTAS

EJERCICIO 01

- Coge la tabla de `matriculas` de la BBDD `escuela`:
 - Y muestra los alumnos que NO SE HAN MATRICULADO A NADA

FUNCIONES DE SQLITE

SON GRATIS

FUNCIONES DE SQLITE

SQLite viene con un montón de funciones que puedes utilizar para solucionararte la vida. Veamos unas cuantas.

FUNCIONES PARA NÚMEROS

FUNCIONES DE SQLITE

NÚMEROS 🙌 ROUND()

- Redondea un número de forma natural.
- Permite 2 parámetros, el número que quieres redondear, y cuantos decimales quieres.

```
-- Redondeos del número 5.7654.
```

```
SELECT ROUND(5.7654), ROUND(5.7654, 1), ROUND(5.7654, 2), ROUND(5.7654, 3);
```

```
/*
```

ROUND(5.7654)	ROUND(5.7654, 1)	ROUND(5.7654, 2)	ROUND(5.7654, 3)
6	5.8	5.77	5.765

```
*/
```

FUNCIONES DE SQLITE

NÚMEROS 👉 CEILING()

- Redondea al alza SIEMPRE.

```
-- Redondeo al alza.
```

```
SELECT CEILING(5.1), CEILING(5.9);
```

```
/*
```

```
+-----+-----+
| CEILING(5.1) | CEILING(5.9) |
+-----+-----+
|           6 |           6 |
+-----+-----+*/
```

FUNCIONES DE SQLITE

NÚMEROS 📌 FLOOR()

- Redondea a la baja SIEMPRE.

```
-- Redondeo a la baja.
```

```
SELECT FLOOR(5.1), FLOOR(5.9);
```

```
/*
```

```
+-----+-----+
| FLOOR(5.1) | FLOOR(5.9) |
+-----+-----+
|          5 |          5 |
+-----+-----+*/
```

FUNCIONES DE SQLITE

NÚMEROS ABS()

- Sirve para obtener el valor absoluto de un número. Es decir, sin el signo.

```
-- Números sin signo.  
  
SELECT ABS(-23), ABS(23);  
  
/*  
+-----+-----+  
| ABS(-23) | ABS(23) |  
+-----+-----+  
|        23 |        23 |  
+-----+-----+*/
```

FUNCIONES DE SQLITE

NÚMEROS RANDOM()

- Devuelve un número aleatorio entre -9223372036854775808 y +9223372036854775807.

```
-- Números aleatorios.
```

```
SELECT RANDOM(), RANDOM(), RANDOM();
```

```
/* +-----+-----+-----+
   | RANDOM()          | RANDOM()          | RANDOM()          |
   +-----+-----+-----+
   | 502213239076796700 | -35725780108435660 | 11593995696926917 |
   +-----+-----+-----+ */
```

FUNCIONES PARA STRINGS

FUNCIONES DE SQLITE

SRINGS LENGTH()

- Devuelve la longitud del valor.

```
-- Longitudes.  
  
SELECT LENGTH('hola'), LENGTH('zapato'), LENGTH(2);  
  
/*  
+-----+-----+-----+  
| LENGTH('hola') | LENGTH('zapato') | LENGTH(2) |  
+-----+-----+-----+  
|           4 |           6 |           1 |  
+-----+-----+-----+*/
```

FUNCIONES DE SQLITE

STRINGS 👉 UPPER()

- Transforma a mayúsculas.

```
-- Todo en mayúsculas. 🧐
```

```
SELECT UPPER('Hola'), UPPER('Zapato'), UPPER(2);
```

```
/*
```

```
+-----+-----+-----+
| UPPER('Hola') | UPPER('Zapato') | UPPER(2) |
+-----+-----+-----+
| HOLA          | ZAPATO          | 2         |
+-----+-----+-----+*/
```

FUNCIONES DE SQLITE

SRINGS LOWER()

- Transforma a minúsculas.

```
-- Todo en minúsculas. 🤪  
  
SELECT LOWER('Hola'), LOWER('ZAPATO'), LOWER(2);  
  
/*  
+-----+-----+-----+  
| LOWER('hola') | LOWER('zapato') | LOWER(2) |  
+-----+-----+-----+  
| hola          | zapato          | 2         |  
+-----+-----+-----+*/
```

FUNCIONES DE SQLITE

SRINGS TRIM()

- **LTRIM** recorta los espacios al principio.
- **RTRIM** recorta los espacios al final.
- **TRIM** recorta los espacios en blancos al principio y al final.

```
-- A recortar espacios. ✂
```

```
SELECT LTRIM('  hola  '), RTRIM('  hola  '), TRIM('  hola  ');
```

```
/*
```

```
+-----+-----+-----+
| LTRIM('  hola  ') | RTRIM('  hola  ') | TRIM('  hola  ') |
+-----+-----+-----+
| 'hola  '          | '  hola'          | 'hola'           |
+-----+-----+-----+*/
```

FUNCIONES DE SQLITE

SRINGS CONCAT()

- Sirve para concatenar los valores que pongamos dentro del **CONCAT** separados por coma.

```
-- Concatenando voy...

SELECT CONCAT('hola', ' ', 'amig@');

/*
+-----+
| CONCAT('hola', ' ', 'amig@') |
+-----+
| hola amig@                    |
+-----+*/
```

Con este podríamos hacer cosas tipo tener un nombre completo o dirección completa mezclando varias columnas.

FUNCIONES PARA FECHAS

FUNCIONES DE SQLITE

SRINGS

DATETIME('NOW'), DATE('NOW'), TIME('NOW')

- Generan fechas automaticamente para AHORA MISMO:
 - **DATETIME('now')** devuelve la fecha y hora actual.
 - **DATE('now')** devuelve la fecha actual.
 - **TIME('now')** devuelve la hora actual.

```
-- Tic tac.....

SELECT DATETIME('now'), DATE('now'), TIME('now');

/*
+-----+-----+-----+
| DATETIME('now') | DATE('now') | TIME('now') |
+-----+-----+-----+
| 2023-08-01 18:24:48 | 2023-08-01 | 18:24:48 |
+-----+-----+-----+*/
```

FUNCIONES DE SQLITE

SRINGS

STRFTIME()

- Extrae el año, mes o día de una fecha.

```
-- Tic tac.....
```

```
SELECT
```

```
  STRFTIME('%Y', '2023-12-30'), -- 2023  
  STRFTIME('%m', 'now'),       -- 08  
  STRFTIME('%d', 'now'),       -- 01  
  STRFTIME('%w', 'now');       -- 2 (Lunes = 0, Martes = 1, Miércoles = 2...)
```

FUNCIONES DE SQLITE

SRINGS

EJEMPLO CON YEAR()

Si quisieras obtener todos los pedidos del año anterior y que siempre sea el anterior al actual, podríamos hacer lo siguiente.

```
-- Obtener los pedidos del año anterior.
SELECT *
FROM matriculas
WHERE STRFTIME('%Y', date) = STRFTIME('%Y' , 'now') - 1;

/*
+-----+-----+-----+-----+
| matricula_id | fecha                | precio_pagado | alumno_id | curso_id |
+-----+-----+-----+-----+
|          5 | 2024-09-15 00:00:00 |         200.00 |          5 |          5 |
+-----+-----+-----+-----+*/
```

FUNCIONES DE SQLITE

FECHAS 🖱️ DATE('FECHA', 'INTERVALO')

- Sirve para añadir o quitar tiempo a una fecha.

```
-- Cambiando calendarios. 📅
```

```
SELECT DATE('2015-05-12', '+1 day'), DATE('2015-05-12', '+2 year');
```

```
/*
```

```
+-----+-----+
| DATE('2015-05-12', '+1 day') | DATE('2015-05-12', '+2 year') |
+-----+-----+
| 2015-05-13 | 2017-05-12 |
+-----+-----+*/
```



